Senai Santo Amaro - Suíço-Brasileira

TicketFest Soluções

Isabella Nascimento Chaves
Seduc - Dest2
Professor Atila Olivi

São Paulo, 2026

TicketFest Soluções

- **Domínio do negócio:** Gestão de eventos, Bilheteria e Controle de Acesso para Festivais de Música e Shows.

- **Contextualização:** A TicketFest é uma plataforma e produtora especializada exclusivamente na comercialização de ingressos e gestão de portarias para shows e festivais de música de médio e grande porte. A empresa atende desde apresentações solo em casas de espetáculo até festivais multimídia com dezenas de atrações e dezenas de milhares de fãs por dia.

- **Problemas enfrentados sem o SGBD:** Antes de adotar um SGBD, a TicketFest controlava suas operações por meio de planilhas eletrônicas compartilhadas por eventos.
    - **Venda Duplicada em Setores Concorridos (*Overbooking*):** Com vendas ocorrendo simultaneamente no site e em pontos de venda físicos no dia do anúncio do show, as planilhas não travavam o estoque em tempo real. O mesmo setor era vendido além da capacidade permitida pela casa de shows ou arena.

    - **Caos e Perda de Margem na Virada de Lotes:** Nos festivais de alta procura, a virada de lote era feita de forma manual. O atraso na atualização das planilhas fazia a empresa vender milhares de ingressos do 1º lote quando já deveria estar cobrando os valores do 2º ou 3º lote, gerando grande prejuízo financeiro.

    - **Filas Quilométricas e Fraudes na Portaria:** No dia do festival, a validação do público era feita buscando nomes em listas impressas ou planilhas locais. O processo era extremamente lento e abria margem para uso de ingressos clonados, prints de tela falsificados e entradas não autorizadas em áreas restritas.

    - **Descontrole em Festivais Multimídia:** Dificuldade em controlar ingressos do tipo "Passaporte" (que dão direito a todos os dias) versus ingressos "Day Pass" (válidos apenas para um dia específico), resultando na entrada de pessoas em dias não contratados.

    - **Vulnerabilidade à LGPD e Vazamentos:** Dados pessoais de milhares de fãs ficavam expostos em pastas da rede sem controle de permissões por perfil ou logs de acesso.

- **Solução com SGBD Relacional:** A implantação de um SGBD Relacional na TicketFest Soluções elimina os problemas operacionais gerados pelas planilhas eletrônicas através dos seguintes mecanismos técnicos:
    - **Eliminação do *Overbooking* via Transações ACID e Locking:** O SGBD garante concorrência segura em picos de vendas através do controle atômico de transações e bloqueio de registros. No momento do pagamento, a cota de ingressos é reduzida com garantia de isolamento, impedindo que o mesmo setor seja vendido acima da lotação máxima do local.

    - **Automação e Consistência na Virada de Lotes:** A transição de preços e lotes é controlada diretamente por regras de integridade e restrições. Ao atingir a cota do 1º lote, o sistema bloqueia novas vendas naquela categoria e direciona o fluxo automaticamente para o lote seguinte.

    - **Validação Antifraude e Agilidade na Portaria:** A tabela de ingressos possui restrição de unicidade para o código QR. O processo de validação na catraca executa uma alteração de estado atômica, impedindo o reuso de ingressos, cópias ou prints de tela.

    - **Controle Rigoroso em Festivais Multimídia:** A modelagem relacional vincula cada ingresso emitido estritamente às categorias de acesso contratadas, permitindo que a validação de acesso bloqueie entradas fora dos dias ou setores permitidos.

    - **Conformidade com a LGPD e Segurança por Perfil (RBAC - Role-Based Access Control):** Os dados dos fãs são centralizados em um ambiente seguro com controle de acesso diferenciado. Operadores de portaria leem apenas a validade do QR Code, enquanto operadores financeiros visualizam dados de vendas, mantendo logs de auditoria e restrições de visibilidade.

    - **Infraestrutura em Nuvem e Alta Disponibilidade com Aiven:** Para suportar a alta concorrência em picos de vendas, o banco de dados db_ticketfest foi implantado na nuvem utilizando a plataforma Aiven. Esta arquitetura assegura alta disponibilidade e acesso remoto distribuído, eliminando a dependência e os gargalos de servidores locais. A comunicação com o SGBD é estritamente protegida por criptografia obrigatória, impedindo a interceptação de tráfego na rede e reforçando a conformidade com as diretrizes de segurança e LGPD adotadas pela empresa.

- **Modelagem Lógica do Banco de Dados:**

- Estrutura das Tabelas, Atributos e Tipos de Dados

1. locais

- **id_local** (INT, PRIMARY KEY, AUTO_INCREMENT): Identificador único do espaço.
- **nome_espaco** (VARCHAR(150), NOT NULL): Nome da arena ou espaço do show.
- **cep** (CHAR(8), NOT NULL): CEP do local.
- **logradouro** (VARCHAR(150), NOT NULL): Logradouro/Rua.
- **numero** (VARCHAR(10), NOT NULL): Número do imóvel.
- **complemento** (VARCHAR(50)): Complemento do endereço.
- **bairro** (VARCHAR(100), NOT NULL): Bairro.
- **cidade** (VARCHAR(100), NOT NULL): Cidade do evento.
- **uf** (CHAR(2), NOT NULL): Estado (UF).
- **capacidade_maxima** (INT, NOT NULL, CHECK (capacidade_maxima > 0)): Lotação máxima permitida.

2. eventos

- **id_evento** (INT, PRIMARY KEY, AUTO_INCREMENT): Identificador do festival/show.
- **id_local** (INT, FOREIGN KEY -> locais.id_local): Local de realização do evento.
- **nome_evento** (VARCHAR(200), NOT NULL): Nome do festival ou show.
- **data_evento** (DATETIME, NOT NULL): Data e horário de início.
- **faixa_etaria** (VARCHAR(20), NOT NULL): Classificação indicativa.
- **status** (VARCHAR(20), NOT NULL, CHECK (status IN ('Agendado', 'Em Andamento', 'Concluido', 'Cancelado'))): Estado do evento.

3. categorias_ingressos

- **id_categoria** (INT, PRIMARY KEY, AUTO_INCREMENT): Identificador do setor/lote.
- **id_evento** (INT, FOREIGN KEY -> eventos.id_evento): Festival ao qual pertence.
- **nome_categoria** (VARCHAR(50), NOT NULL): Descrição do setor/lote (ex: Passaporte 3 Dias, Pista Premium Lote 1).
- **preco** (DECIMAL(10,2), NOT NULL, CHECK (preco >= 0)): Valor unitário do ingresso.
- **quantidade_disponivel** (INT, NOT NULL, CHECK (quantidade_disponivel >= 0)): Cota de ingressos do lote.

4. clientes

- **id_cliente** (INT, PRIMARY KEY, AUTO_INCREMENT): Identificador do fã/comprador.
- **nome** (VARCHAR(200), NOT NULL): Nome completo.
- **cpf** (VARCHAR(14), UNIQUE, NOT NULL): CPF do comprador.
- **email** (VARCHAR(150), UNIQUE, NOT NULL): E-mail para recebimento do ingresso.
- **telefone** (VARCHAR(15), NOT NULL): Telefone de contato.

5. compras

- **id_compra** (INT, PRIMARY KEY, AUTO_INCREMENT): Identificador do pedido.
- **id_cliente** (INT, FOREIGN KEY -> clientes.id_cliente): Comprador responsável.
- **data_compra** (DATETIME, DEFAULT CURRENT_TIMESTAMP): Data e hora do pagamento.
- **valor_total** (DECIMAL(10,2), NOT NULL, CHECK (valor_total >= 0)): Valor total do pedido.
- **forma_pagamento** (VARCHAR(30), NOT NULL, CHECK (forma_pagamento IN ('PIX', 'Cartao Credito', 'Boleto'))): Método de pagamento.
- **status_pagamento** (VARCHAR(20), NOT NULL, CHECK (status_pagamento IN ('Aprovado', 'Pendente', 'Cancelado'))): Estado do pagamento.

6. ingressos_emitidos

- **id_ingresso** (INT, PRIMARY KEY, AUTO_INCREMENT): Identificador único do bilhete individual.
- **id_compra** (INT, FOREIGN KEY -> compras.id_compra): Pedido de origem.
- **id_categoria** (INT, FOREIGN KEY -> categorias_ingressos.id_categoria): Categoria/setor do ingresso.
- **codigo_qr** (VARCHAR(100), UNIQUE, NOT NULL): Código/Hash único de leitura na catraca.
- **status_utilizacao** (VARCHAR(20), DEFAULT 'Valido', CHECK (status_utilizacao IN ('Valido', 'Utilizado', 'Cancelado'))): Estado do ingresso na portaria.

- **Relacionamentos e Cardinalidades**

- **locais para eventos (1: N):** Um local pode sediar múltiplos festivais ao longo do tempo, mas cada festival ocorre em apenas um local específico.

- **eventos para categorias_ingressos (1: N):** Um festival possui vários setores e lotes de ingressos; cada categoria de ingresso é vinculada a apenas um festival.
- **clientes para compras (1: N):** Um fã pode realizar várias compras na plataforma; cada compra é associada a apenas um cliente titular.
- **compras para ingressos_emitidos (1: N):** Uma compra/pedido pode conter um ou mais ingressos emitidos; cada ingresso individual pertence a um único pedido.
- **categorias_ingressos para ingressos_emitidos (1: N):** Uma categoria/lote pode gerar centenas de ingressos individuais emitidos; cada ingresso pertence a apenas uma categoria.

Aplicação das Formas Normais

Primeira Forma Normal (1FN)

• **Regra:** Todos os atributos devem conter apenas valores atómicos (indivisíveis) e a tabela não deve conter grupos repetitivos ou colunas multivaloradas.

• **Aplicação na TicketFest:**

- **Eliminação de Grupos Repetitivos:** Em vez de armazenar uma lista de ingressos adquiridos numa única linha da tabela compras, criou-se a entidade ingressos_emitidos. Cada linha nesta tabela representa estritamente um único ingresso individual.
- **Atributos Atómicos:** Atributos como cpf, email, telefone e data_evento contêm valores únicos. O endereço da tabela locais foi atomizado nos campos cep, logradouro, numero, complemento, bairro, cidade e uf.

Segunda Forma Normal (2FN)

• **Regra:** A tabela deve estar na 1FN e todos os atributos não-chave devem depender da totalidade da chave primária (eliminação de dependências funcionais parciais em chaves compostas).

• **Aplicação na TicketFest:**

- **Uso de Chaves Primárias Simples:** Todas as tabelas do modelo utilizam chaves primárias numéricas simples auto-incrementáveis (id_local, id_evento, id_categoria, id_cliente, id_compra, id_ingresso). Como não existem chaves primárias compostas, não há possibilidade de dependência parcial.
- **Separação de Entidades:** O nome da categoria de ingresso (nome_categoria) e o seu preço (preco) dependem unicamente do id_categoria. Portanto, esses dados residem exclusivamente na tabela categorias_ingressos e não na tabela ingressos_emitidos.

Terceira Forma Normal (3FN)

• **Regra:** A tabela deve estar na 2FN e nenhum atributo não-chave pode depender de outro atributo não-chave (eliminação de dependências transitivas: $A \rightarrow B \rightarrow C$).

• **Aplicação na TicketFest:**

- **Relação Eventos e Locais:** Na tabela eventos, armazena-se apenas o identificador id_local. Atributos como nome_espaco, capacidade_maxima ou cidade pertencem estritamente à tabela locais. Colocar esses campos em eventos geraria uma dependência transitiva (id_evento → id_local → capacidade_maxima).
- **Relação Compras e Clientes:** A tabela compras guarda apenas a chave estrangeira id_cliente. Dados cadastrais do fã (nome, cpf, email) ficam restritos à tabela clientes, evitando redundância caso o mesmo cliente realize múltiplas compras.

Quarta Forma Normal (4FN)

• **Regra:** A tabela deve estar na 3FN e não deve conter duas ou mais dependências multivaloradas independentes na mesma entidade.

• **Aplicação na TicketFest:**

- Um festival pode ter múltiplos artistas/atrações e múltiplos patrocinadores associados de forma independente. Se a tabela eventos tentasse armazenar listas de atrações e de patrocinadores na mesma estrutura, haveria proliferação de linhas redundantes (produto cartesiano). A modelagem previne esse problema isolando os dados de cada entidade em estruturas próprias.

- **Como Gerar o Dump (Exportar Estrutura e Dados)**

1. Abra o **MySQL Workbench** e conecte-se ao seu servidor de banco de dados.
2. No menu superior, acesse **Server** > **Data Export**.
3. Na lista de bancos de dados, selecione a base db_ticketfest.
4. Em **Export Options**, marque a opção **Export to Self-Contained File** e defina o local onde o arquivo será salvo (ex: dump_ticketfest.sql).
5. No menu suspenso do canto superior direito, garanta que a opção **Dump Structure and Data** está selecionada.
6. Clique no botão **Start Export** no canto inferior direito para concluir a geração do arquivo.

- **Como Restaurar o Dump (Importar Estrutura e Dados)**

1. Abra o **MySQL Workbench** no ambiente de teste ou demonstração.
2. No menu superior, acesse **Server** > **Data Import**.
3. Selecione a opção **Import from Self-Contained File** e localize o arquivo dump_ticketfest.sql.
4. Em **Default Target Schema**, clique no botão **New...** e crie a base de dados com o nome db_ticketfest.
5. Acesse a aba **Import Progress** no canto inferior e clique em **Start Import** para carregar todas as tabelas e dados.

- **Diagrama EER**

